

USC University of Southern California

-

Team Members

Name
ATHISH RAJ MOHAN
CHIRAG SRINIVAS

“🌎 World Bank and Human Development Index Data Analysis Dashboard”

1. Project Overview:

1.1 Abstract:

This project introduces an interactive analytics dashboard for exploring global economic and social development trends. A custom-built Pandas-style engine in Python was designed to perform filtering, projection, aggregation, group by and joins. Large-scale datasets are processed using chunk-based parsing with a generator-driven approach. The dashboard visualizes GDP growth, GDP per capita, HDI distribution across various economic groups, and average life expectancy per region. It highlights how database concepts and data engineering techniques transform raw data into actionable insights.

1.2 Motivation:

The motivation was to bridge the gap between raw development data and meaningful, accessible insights. By manually implementing core operations, the project deepens understanding of database engine internals. It integrates data engineering principles with interactive visualization and web application development. The platform enables dynamic exploration of indicators in a user-friendly and engaging environment. Ultimately, it connects theoretical database concepts with practical applications in policy, and business.

2. Algorithms:

2.1 Data Loading, Parsing and DataGrid Construction with and without Chunking:

This process transforms raw CSV files into structured DataGrid objects using the parseCSV() function with optional chunk-based processing. The DataGrid object is internally storing the data in the form of a dictionary with the keys being column names and the values being a list of row values. Without chunking, the entire file is loaded into memory, parsed with automatic type inference and missing value handling, and returned as a single DataGrid. With chunking, the file is processed in fixed-size batches using a generator to produce multiple DataGrid objects for memory-efficient large-scale processing. Each DataGrid ensures relational consistency and supports SQL-like operations such as filtering, projection, aggregation, group by, and joins. Streamlit caching is used to avoid repeated disk access and improve performance.

Algorithm:

- Call parseCSV(filePath) for non-chunk loading OR parseCSV(filePath, chunk_size) for chunk-based loading.
- Open CSV file in read mode, read the first line as the header.
- Split the header to extract column names and initialize an empty dictionary with column names as keys.
- If chunk_size is not given:
  - Load the entire CSV file into memory
- If chunk_size is provided:
  - Initialize row_counter = 0, a generator for chunk processing and to read the file incrementally.
- For each row:
  - Split rows by comma.
  - If values < columns → pad with None
  - If values > columns → truncate the extra values

        - Infer the correct data type of each value using inferType helper function
        - Append the processed value to the corresponding columns list
        - Increment row counter (only in chunk mode)
    - For chunk mode, If row_counter >= chunk_size:
        - Construct a DataGrid from the dictionary
        - Yield the DataGrid chunk
        - Reset dictionary and counter
    - If remaining rows exist (in chunk mode), yield the final DataGrid.
    - In non-chunk mode, construct and return a single DataGrid and close the file.

2.2 Projection Operation with and without Chunking: Projection selects specific columns from the dataset using the overloaded __getitem____ method. Without chunking, columns are extracted directly from memory, while with chunking, projection is applied to each batch independently, and are merged to obtain the final result.

Algorithm:

- Input filtered DataGrid (or chunk list). User specifies a list of required columns
- If without chunking:
    - __getitem__() extracts selected columns directly
    - Construct and return a new projected DataGrid.
- If with Chunking:
    - Apply projection independently on each chunk and collect all the projected chunks. Now, use the merge_chunks() function to combine column-wise values from all projected chunks and return the final projected DataGrid.

2.3 Filtering Operation with and without Chunking: Filtering selects rows using Boolean conditions that satisfies a particular specified condition. In non-chunk mode, filtering is done in a single pass over the full dataset in memory. In chunk mode, filtering is applied to each batch and the results are merged for scalability.

Algorithm:

- Input DataGrid, and the filtering condition specified by the user
- Based on the operators specified by the users, a columns of Booleans is returning conveying the value is being satisfied or not satisfied the condition
- If without chunking:
    - DataGrid.__getitem__() detects the Boolean mask → filter_by_mask() is applied → Rows with True values are retained → Return a new filtered DataGrid.
- If with chunking:
    - Load dataset using chunking → Initialize an empty result list → For each chunk: Apply the filtering condition and Store the filtered chunk.
- Use merge_chunks() to combine all the filtered chunks and return DataGrid.

2.4 Join Operation - with and without Chunking: The join operation combines two DataGrids based on a common key column using inner, left, right, or outer join semantics. In non-chunk mode, an index (hash map) is created on the join key of the right DataGrid to allow fast lookup of matching rows for each left row, improving performance compared to a full nested scan. Matching rows are merged, and unmatched rows are handled according to the join type. In chunk mode, one or both DataGrids are processed in batches; each left chunk is merged with all right chunks independently, and partial results are combined using merge_join_chunks() to produce the final joined DataGrid.

Algorithm:

- Input: Two DataGrids (dg1, dg2), join key (on), and join type (how).
- **If without chunking:**
- - Convert both DataGrids into row-wise dictionaries.
  - Build a hash-based index on the right DataGrid keyed by the join column.
  - Iterate over each row in the left DataGrid and find matching rows in the right DataGrid using the index.
  - Merge matching rows and handle unmatched rows according to the join type.
  - Construct and return the final merged DataGrid.
- **If with chunking:**
  - Load the DataGrids in chunks.
  - For each left chunk, merge with all right chunks independently using the right table index for fast lookups.
  - Keep track of unmatched rows for left or outer joins to ensure they are included with None values in the missing columns
  - Combine all partial merged chunks using merge_join_chunks() to produce the final joined DataGrid.

2.5 Aggregation Operation - with and without Chunking: Aggregation computes summary statistics (like sum, min, max, count, avg) on one column of a DataGrid. In non-chunk mode, the aggregation is applied directly to the full column in memory. In chunk mode, each chunk computes partial aggregates independently, which are then merged to produce the final result.

Algorithm:

- Input: DataGrid, column name and aggregation function (sum, min, max, count, avg)
- **If without chunking:**
- - Access the target column using the Column class.
  - Apply the aggregation function directly on the column values using the corresponding method
  - Return a new DataGrid containing the aggregated results.

- **If with chunking:**
  - Load the dataset in chunks.
  - For each chunk: compute partial aggregates for each column according to the specified function. For sum, min, max, count: calculate the partial values directly. For avg: store both partial sum and count for correct final averaging.
  - Use merge_simple_agg_chunks() to combine all partial results into a final aggregated DataGrid.

2.6 Group By Operation - with and without Chunking: Group By aggregation first groups rows by one or more columns and then applies aggregation functions (sum, min, max, count, avg) on other columns within each group. Chunking allows processing large datasets by computing partial group aggregates per chunk and merging them later.

Algorithm:

- Input: DataGrid, group-by columns, and aggregation dictionary specifying columns and functions.
- **If without chunking:**
- - Partition the rows into groups based on the unique values of the specified group-by columns.
  - For each group, apply the aggregation functions to the other columns using the corresponding Column methods.
  - Construct a new DataGrid with group keys and aggregated results.
- **If with chunking:**
  - Load the dataset in chunks.
  - For each chunk:
    - For each row, determine its group key (tuple of values from group-by columns).
    - Maintain a partial aggregation per group:
      - sum, min, max, count: accumulate values.
      - avg: store partial sum and count for correct final averaging.
  - Use merge_groupby_chunks() to combine all partial results into a DataGrid containing the grouped keys and aggregated results.

3. DATASETS AND APPLICATION

3.1 DATASETS:

World Bank Economic Indicators Dataset:

The World Bank Economic Indicators dataset contains country-wise yearly data on key economic metrics such as GDP, GDP per capita, inflation, population, and trade indicators. It is compiled from official national statistics and international agencies. This dataset is widely used for analyzing global economic trends.

Human Development Index (HDI) Dataset:

The HDI dataset, published by the United Nations Development Programme (UNDP), provides a composite measure of human development based on health, education, and income indicators. It assigns an HDI score to each country to reflect overall quality of life. This dataset is commonly used to compare development levels and study socio-economic progress across nations.

3.2 APPLICATION

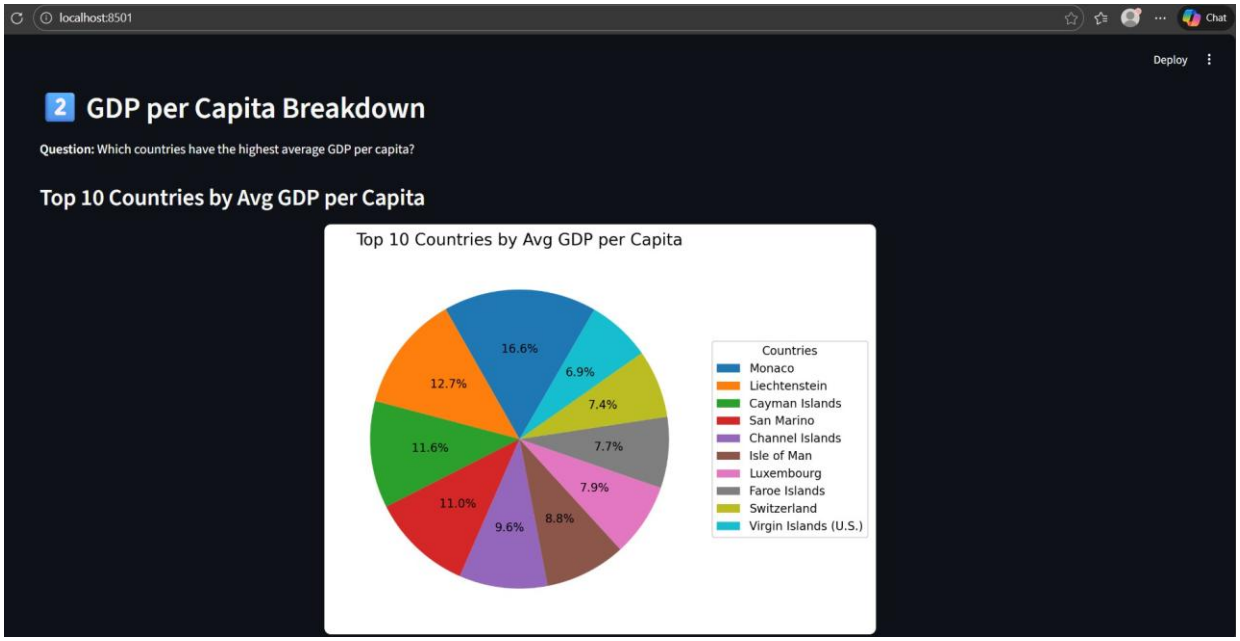
The application is an interactive Streamlit-based dashboard that allows users to explore global economic and social indicators using a custom-built data processing engine. It supports filtering, projection, aggregation, group by and joins, and visualization of large datasets through an efficient chunk-based processing mechanism. The tool enables insight-driven analysis of GDP, HDI, and other development indicators for research and decision-making.

a) GDP Growth over time - **Question: How has GDP changed for a selected country over the years? -** This visualization shows the year-wise GDP trend for a user-selected country using an interactive dropdown. The dataset is first loaded using our custom CSV parser and filtered based on the selected country using Boolean masking. The filtered GDP and Year columns are extracted from the DataGrid. A Matplotlib line chart is then generated and rendered dynamically in the Streamlit interface.

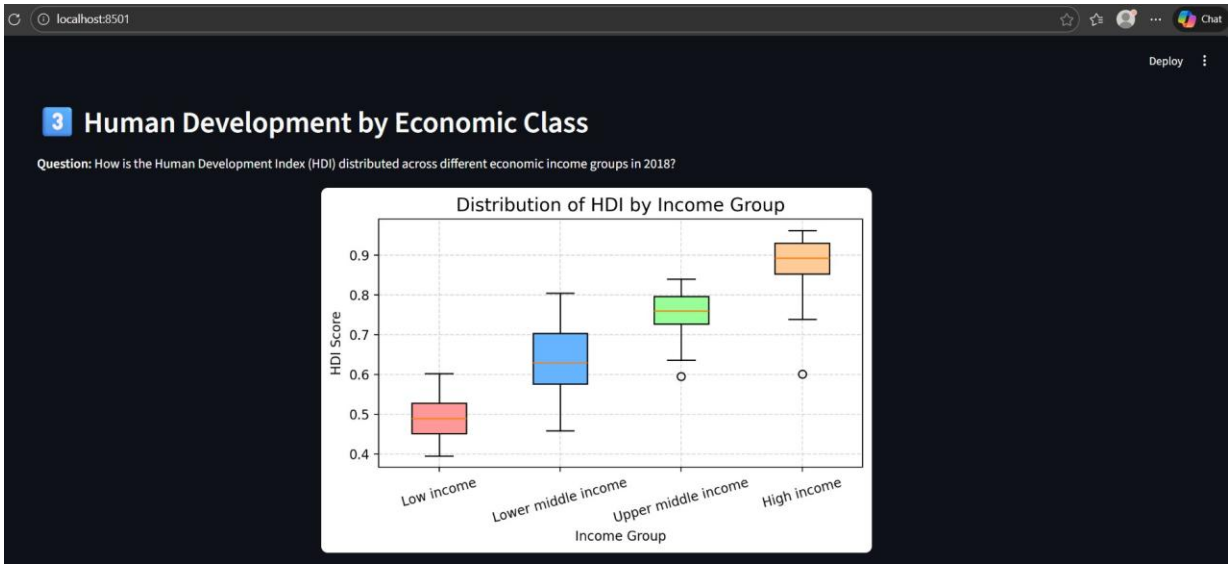


b) GDP per Capita Breakdown - **Question: Which countries have the highest average GDP per capita? -** This visualization identifies the top 10 countries with the highest average GDP per capita using chunk-based processing. The World Bank dataset is loaded in chunks of 5000 rows for memory efficiency. Each chunk is filtered, grouped, and aggregated independently using the custom groupby and avg functions. The partial

results are merged using merge_groupby_chunks, and the final output is displayed as a pie chart.



c) Human Development by Economic Class - **Question: How is the Human Development Index distributed across different economic income groups in 2018? -** This visualization shows how Human Development Index (HDI) values vary across different income groups for the year 2018. The World Bank and HDI datasets are joined using a custom inner join on the Country Code column. Countries are grouped into income categories such as low, middle, and high income. A box plot is generated to visualize the HDI distribution across these economic classes.



d) Average Life Expectancy by Region - **Question: How does life expectancy vary across different parts of the world?** - This visualization compares the average life expectancy across world regions for a selected year using a dropdown. The dataset is filtered by year using the custom filtering logic. Regional averages are computed using the group by and aggregation functions built into the DataGrid. The results are displayed as a bar chart with values annotated for clear comparison across regions.



4. LEARNING EXPERIENCES:

We gained hands-on experience in building a pandas-like data pipeline without relying on Pandas, learning how raw CSV data is transformed into structured formats through custom parsing, type inference, and memory management. Implementing the DataGrid class deepened our understanding of tabular structures, including column storage, row alignment, and indexing. We developed core operations such as filtering, projection, aggregation, grouping, and joins using low-level logic, while Boolean mask-based filtering revealed how queries execute internally. Chunk-based and non-chunk pipelines highlighted scalability and efficiency, and handling missing values, inconsistent formats, and mixed types improved our data cleaning skills. Overall, the project strengthened our grasp of modular design, reusable components, and performance constraints in real-world systems.

5. Contribution of Team Members

5.1 Contribution of Athish Raj Mohan:

I was responsible for implementing one of the core functionalities of the project, which included the basic CSV parser along with projection and filtering operations. I developed full chunk-based batch processing support for CSV parsing, filtering, projection, and the merging of chunks to enable efficient handling of large datasets. I collaborated with Chirag on dataset selection and contributed to the overall application UI design. Further,

along with Chirag, I integrated all the implemented functions into the application, and I performed data analysis to generate two visualizations: (1) the change in GDP of a selected country over the years, and (2) the comparison of countries with the highest average GDP per capita. Finally, Chirag and I carried out the final polishing, debugging, and testing of the complete system to ensure correctness and smooth performance.

5.2 Contribution of Chirag Srinivas:

I was responsible for implementing a few of the core operations which are aggregation, group by and join. I built the chunk-based support for aggregation, group by, join, and the merging of chunks to enable efficient handling of large datasets. I collaborated with Athish on dataset selection and contributed to the overall application UI design. Further, along with Athish, I integrated all the self implemented functions into the application, and I performed data analysis to generate two visualizations: (1) the distribution of human development index across various economic groups, and (2) comparison of average life expectancies of each region. Finally, Athish and I carried out the final polishing, debugging, and testing of the complete system to ensure correctness and smooth performance.